

CareLite Healthcare Learning Platform

Product Requirements Document

Small healthcare SaaS project for learning microservices, tenant onboarding, AWS, and DevOps patterns.

Field	Value
Document Type	PRD
Version	0.1
Date	2026-05-15
Status	Draft for learning and implementation planning
Primary Stack	Java 21, Spring Boot 3.3.x, Spring Cloud 2023.0.x, Maven 15-module monorepo, Spring Cloud Gateway, OpenFeign, Amazon MSK/Kafka, MongoDB Atlas, AWS EKS, Cognito, S3, WAF, Shield, Jenkins, Argo CD

Document Control

Item	Details
Owner	CareLite learning project team
Audience	Developer, DevOps engineer, solution architect, product owner
Purpose	Guide a small healthcare platform that teaches production microservice patterns.
Scope Level	MVP plus platform learning patterns

1. Executive Summary

CareLite is a small healthcare domain project designed to learn production-grade microservice, tenant onboarding, cloud, DevOps, and observability patterns without the size of a full hospital ERP. The product serves small clinics that need patient registration, doctor scheduling, appointments, basic prescriptions, billing, document storage, and audit history.

Product stance

Keep the healthcare workflow intentionally small. The main objective is to practice patterns from a larger SaaS architecture: schema-based multi-tenancy, OAuth2, S3 documents, outbox events, sagas, audit logs, gateway filters, rate limits, circuit breakers, CI/CD, GitOps, and EKS deployment.

2. Product Goals

- Provide a clear learning path for building secure Spring Boot microservices.
- Support automatic onboarding of new clinic tenants with schema-based data isolation.
- Use AWS Cognito for OAuth2 Authorization Code with PKCE.
- Use S3 for tenant-isolated document storage.
- Use Spring Cloud Gateway for route-level policies, tenant filters, OAuth2 resource server checks, and Redis-backed rate limits.
- Use Feign clients with Resilience4j timeouts, retries, circuit breakers, and fallbacks for synchronous service calls.
- Use Kafka for asynchronous domain events and MongoDB as the event store for audit and replay learning.
- Deploy services to AWS EKS using Jenkins for CI and Argo CD for CD.
- Protect the public edge with Route 53 DNS, CloudFront CDN, ACM TLS certificates, AWS WAF rate-based rules, and ALB ingress.
- Use open-source observability with Prometheus, Grafana, Loki, Tempo, and OpenTelemetry.

2.1 Verified Technology Baseline

Category	Baseline
Languages and frameworks	Java 21 with virtual threads and records; Spring Boot 3.3.x; Spring Cloud 2023.0.x; Maven multi-module with 15 modules.
Databases and storage	Amazon RDS PostgreSQL Multi-AZ with per-tenant schemas; MongoDB Atlas for event store and CQRS read models; ElastiCache Redis for cache, locks, and gateway rate limiter; Amazon S3 for files and cold archives.

Category	Baseline
Messaging	Amazon MSK/Kafka with 48 partitions for domain.events; Apache Avro payload contracts; Confluent Schema Registry; Transactional Outbox pattern.
Identity and security	AWS Cognito OAuth2 with custom JWT claims; Spring Security RBAC; IAM and IRSA; AWS WAF and Shield; Falco runtime monitoring; Trivy CVE scanning.
Architecture patterns	Event Sourcing v1.2 learning path, CQRS, Saga orchestration, Outbox, idempotency, per-schema multi-tenancy, DDD bounded contexts.
Observability	Prometheus, Grafana, Loki, Fluent Bit, OpenTelemetry, Tempo, Alertmanager, and Grafana Faro for browser telemetry.
Batch and documents	Spring Batch with 5 learning jobs; iText 8 for PDF; Apache POI for Excel; EventArchivalJob writes MongoDB event-store data to S3 Avro nightly.
CI/CD and quality	Jenkins, Argo CD GitOps, Google Jib, SonarQube, Flyway, Testcontainers, ArchUnit, and WireMock.

3. Target Users

Persona	Needs	Primary Capabilities
Platform Admin	Onboard clinics and monitor system health.	Create tenants, suspend tenants, view audit history.
Clinic Admin	Manage clinic setup and staff.	Manage doctors, receptionists, clinic settings, documents.
Doctor	See appointments and write notes.	View schedule, update visit notes, issue prescription.
Receptionist	Manage front-desk operations.	Register patients, book appointments, create invoices.
Patient	View own appointments and documents.	View appointment status, download prescription or invoice.

4. MVP Scope

4.1 Included

- Clinic tenant onboarding with automatic PostgreSQL schema creation.
- Cognito user and group setup for tenant users.
- Patient registration and profile management.
- Doctor profile and schedule slots.
- Appointment booking, cancellation, and completion.
- Basic prescription or doctor note capture.
- Invoice creation and payment status tracking.
- S3-backed document metadata and pre-signed upload/download flow.
- Domain event publishing through the outbox pattern.
- Audit event timeline backed by MongoDB for tenant admins.
- Route, tenant, and role-aware rate limiting at CloudFront/WAF and Spring Cloud Gateway.

- Highly available EKS deployment baseline with at least two replicas for stateless services.

4.2 Out of Scope for First Version

- Electronic health record integrations such as HL7 or FHIR.
- Insurance claim adjudication.
- Real payment gateway settlement.
- Pharmacy inventory and lab management.
- Advanced clinical decision support.
- Native mobile applications.

5. Services

Service	Responsibility	Primary Data
api-gateway	Spring Cloud Gateway routing, OAuth2/JWT validation, tenant resolution filter, route-level rate limiting, circuit breaker filters.	No business data; Redis rate limiter state.
identity-service	Cognito OAuth2 integration, custom claims, group mapping, RBAC synchronization.	Master identity metadata.
tenant-service	Tenant onboarding, schema provisioning, Cognito group/user setup.	Master schema.
clinic-service	Patients, doctors, appointments, prescriptions, document metadata.	Tenant schema.
billing-service	Invoices, payment status, billing saga events.	Tenant schema.
document-service	PDF/Excel generation, S3 document upload/download, cold archive handoff.	S3 and tenant schema metadata.
batch-service	Spring Batch jobs for bulk import, reminders, invoice export, replay, and event archival.	Tenant schema, MongoDB, S3.
audit-event-service	Kafka event ingestion, audit API, replay experiments.	MongoDB event store collections.

5.1 Platform Capabilities

Capability	Product Requirement
Edge routing	Route 53 routes tenant domains to CloudFront or ALB; ACM provides TLS certificates.
Edge rate limit	Use AWS WAF rate-based rules on CloudFront and ALB. Route 53 itself does DNS routing, not HTTP request throttling.
Gateway rate limit	Spring Cloud Gateway uses RedisRateLimiter with policies per route, tenant, role, and client IP.
Resilience	OpenFeign clients use Resilience4j circuit breakers, timeouts, bulkheads, retries, and fallbacks.
Event store	Audit events are persisted to MongoDB with tenant, aggregate, actor, and time-based indexes.
Schema registry	Avro schemas are registered in Confluent Schema Registry and versioned with backward compatibility rules.
High availability	EKS services run across multiple Availability Zones with HPA, readiness probes, PDBs, and rolling updates.

6. Success Metrics

Metric	Target
Tenant onboarding	New clinic ACTIVE within 60 seconds in local/dev environment.
Patient registration API	p95 below 300 ms excluding cold starts.
Appointment booking	Idempotent retry returns same appointment for same key.
Audit event availability	Event visible in audit API within 10 seconds.
Deployment lead time	Commit to EKS sync through Jenkins and Argo CD below 10 minutes.
Availability target	Learning target of 99.9% for stateless application services in dev/stage design.
Rate limiting	Gateway returns 429 for route or tenant quota breaches and emits metrics by tenant.
Observability	Every request has logs, metrics, and traces correlated by trace ID.
Kafka scale	domain.events is designed with 48 partitions and tenant+aggregate partition keys.
Quality gates	Jenkins pipeline fails on unit/integration test failure, SonarQube quality gate failure, or blocking Trivy finding.

7. Key Product Requirements

- Every request must carry tenant context after the gateway resolves the tenant.
- Tenant data must be isolated by PostgreSQL schema.
- All create operations that can be retried must support Idempotency-Key.
- Clinical documents must be stored in S3 under a tenant-specific prefix.
- Authentication must use Cognito OAuth2 and JWT validation.
- Public traffic must enter through Route 53, CloudFront/WAF or ALB/WAF, ACM TLS, Kubernetes ingress, and then api-gateway.
- Gateway must provide a tenant gateway filter, route predicates, rate limiting, circuit breaker filters, and correlation ID propagation.
- Internal REST calls must use Feign clients with explicit timeout and circuit breaker policies.
- Kafka must carry domain events; MongoDB must store the immutable audit/event stream.
- MSK topic domain.events must start with 48 partitions and Avro schemas managed through Confluent Schema Registry.
- MongoDB Atlas must hold event store and CQRS read-model collections; cold archival must write S3 Avro files nightly.
- Spring Batch must include five learning jobs: PatientBulkImportJob, AppointmentReminderJob, InvoiceExportJob, ReplayJob, and EventArchivalJob.
- PDF and Excel features must use iText 8 and Apache POI respectively.
- Quality engineering must include Flyway, Testcontainers, ArchUnit, WireMock, SonarQube, Google Jib, and Trivy.
- Audit history must capture actor, tenant, event type, aggregate, timestamp, and correlation ID.
- CI/CD must separate Jenkins CI from Argo CD GitOps deployment.

8. Product Risks

Risk	Impact	Mitigation
Tenant provisioning partially fails.	Tenant stuck between schema, Cognito, and S3 setup.	Provisioning state machine with retry and compensation markers.
Schema-per-tenant becomes hard to migrate.	Operational complexity grows.	Flyway migration runner with tenant registry and version tracking.
Cognito claims drift from tenant database.	Authorization errors.	Tenant-service owns user sync and emits audit events for updates.
Route 53 is mistaken for a rate limiter.	False security assumption.	Use AWS WAF rate-based rules on CloudFront and ALB; keep Route 53 for DNS health routing.
MongoDB event store grows quickly.	Storage and query costs rise.	Partition/index by tenant, event type, aggregate, and time; archive cold events to S3 Avro nightly.
Schema evolution breaks consumers.	Kafka consumers fail at runtime.	Use Avro plus Schema Registry with compatibility checks in CI.
Circuit breaker settings hide real failures.	Operational blind spots.	Expose breaker state and fallback counts to Prometheus/Grafana.
Too much platform scope.	Learning slows down.	Keep only five services and add complexity phase by phase.

9. Roadmap

Phase	Outcome
1	Maven 15-module monorepo, Java 21, Spring Boot 3.3.x, Spring Cloud 2023.0.x, local Docker Compose.
2	Tenant onboarding with PostgreSQL schema provisioning and Flyway.
3	Spring Cloud Gateway, Cognito OAuth2, RBAC, tenant propagation, Redis route rate limits.
4	Clinic domain APIs: patient, doctor, appointment, prescription.
5	Billing service, Feign calls, Resilience4j circuit breakers, outbox, MSK/Kafka, Avro, Schema Registry, MongoDB audit service.
6	S3 document flow, iText 8 PDFs, Apache POI Excel, five Spring Batch jobs, EventArchivalJob to S3 Avro.
7	Jenkins, Google Jib, SonarQube, Trivy, ECR, Helm, Argo CD, EKS deployment with ingress, ACM, Secrets Manager.
8	CloudFront/WAF/Shield edge controls, open-source observability dashboards, Grafana Faro, alerts, HPA and HA hardening.